

# Technical Report Guideline — SPY Market Regime Research

## Section IV.6 (Research Architecture) and Appendix / Reproduction

*Owner: Pipeline*

Generated from the live repository state, 2026-09-04

## 1 IV.6 Research Architecture

### 1.1 Design rationale: why two implementation paths exist, and what the risk of that choice actually cost

**Question:** What does the end-to-end research pipeline look like, and why is it built the way it is rather than as one linear script?

**Method:** Read the current source tree directly (E1-S1 through E4-S2), the git history of when each stage was added, and the independent audit reports in docs/ rather than relying on any single stage's own self-description.

**Evidence:** Two execution paths currently coexist for the E1/E2 core. The first is a sequence of standalone, single-purpose scripts (E2-S1\_Baseline\_Zero\_Predictor through E2-S4\_Generate\_Canonical\_OOS\_Prediction\_Table), each with its own README.md, output/, and tests/, explicitly reusing E2-S1's splits.py and metrics.py rather than reimplementing them. The second, E3-S2\_Data\_Model\_Integration\_Flow/, is a unified pipeline: pipeline\_config.yaml is the single declared source of truth for every parameter, pipeline/run\_pipeline.py is the single command-line entry point, and every stage's inputs and outputs are chained by SHA-256 (StageContract in pipeline/contract.py), refusing to reuse a stage's output if its recorded input hash no longer matches what is currently on disk.

**Interpretation:** This dual-path design was not neutral. This cycle, an independent audit (docs/E2-S2\_LightGBM\_single\_config\_audit\_report.md) found that the two implementations of the LightGBM training stage had silently diverged: the standalone script (train\_lightgbm.py) hard-codes random\_state=42, n\_jobs=-1 in its parameter dictionary, while the pipeline's YAML-sourced parameters had neither key. Because subsample=0.8 and colsample\_bytree=0.8 are both below 1.0, LightGBM genuinely subsamples rows and columns every boosting round, so the seed is not a cosmetic setting — it changes which model gets fit. The two paths therefore produced two different, but each individually fully deterministic, LightGBM runs (MAE 0.0160559 with the seed vs. 0.0160734 without it, on the historical data snapshot), and this eventually surfaced as two of the project's own downstream tables (RQ2's regime table and RQ3's model-ranking table) reporting two different LightGBM MAE values at the same point in time, with no existing check able to detect the disagreement. The lesson generalizes beyond this one bug: any two independently-maintained implementations of "the same" logic are a standing invitation for silent divergence, and the fix that matters structurally is not merely correcting the one parameter but eliminating the

duplication as the source of drift (Appendix A.6 walks through the specific fix and the regression tests now guarding against recurrence).

## 1.2 Pipeline diagram

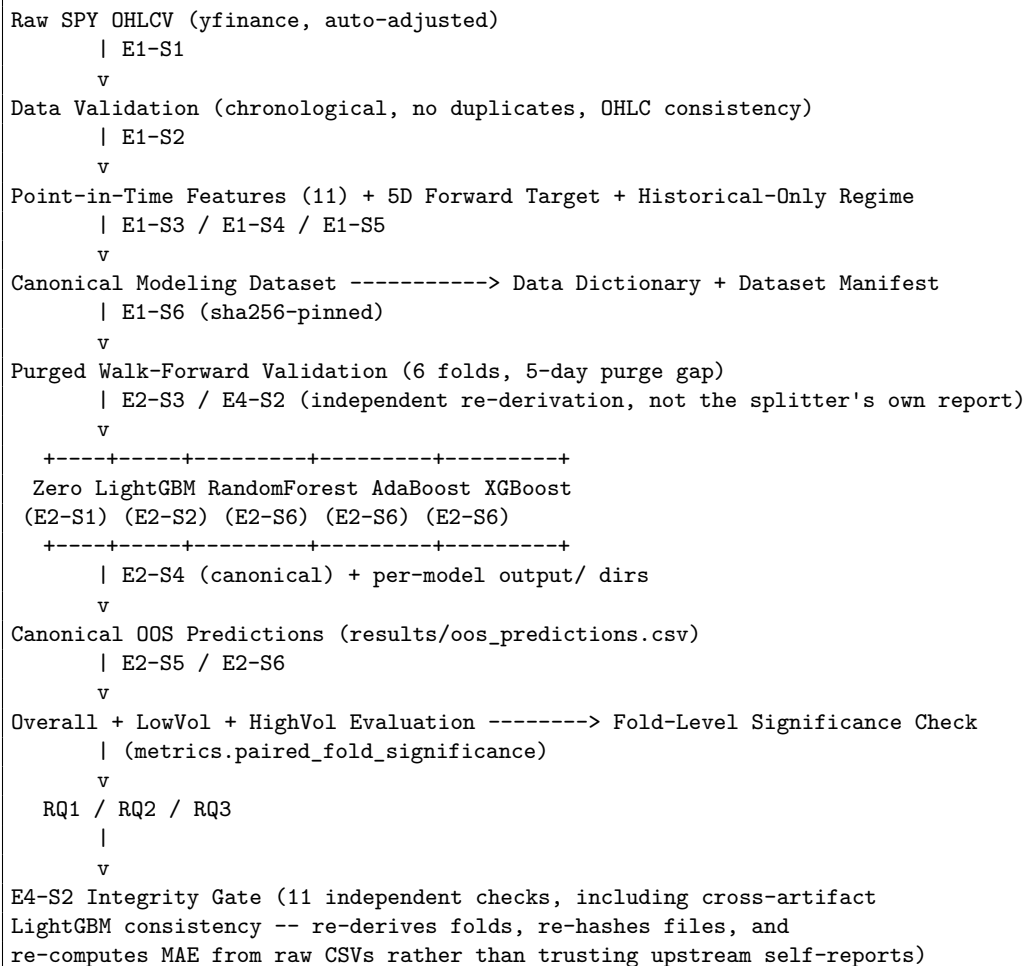


Figure 1: End-to-end research architecture, as implemented. Every arrow is a file on disk with a recorded SHA-256, not an in-memory handoff.

## 1.3 Independent verification is structurally separated from generation

**Question:** Why do E4-S1 (pre-model leakage gate) and E4-S2 (OOS integrity gate) exist as separate modules rather than as assertions inside the pipeline stages themselves?

**Method:** Compare what each gate re-derives against what it merely reads.

**Evidence:** E4-S1 rebuilds every feature point-in-time from the raw OHLCV artifact without importing the notebook that produced the canonical dataset, and requires the published values to match at `rtol=1e-9`. E4-S2 independently re-derives the six fold boundaries from `splits.py` applied to the canonical dataset's own dates — it does not read E2-S3's reported fold boundaries and trust them. This cycle, E4-S2 was extended with a sixth check that re-

hashes `results/oos_predictions.csv`'s declared source file and re-computes LightGBM's aggregate MAE directly from the raw prediction CSVs, rather than reading any stage's self-reported summary statistic.

**Interpretation:** A gate that reads a stage's own summary and re-prints it is not an independent check; it is a formatting step. Every one of E4-S1 and E4-S2's checks is designed so that the object under audit cannot pass by construction alone — it has to independently agree with a from-scratch recomputation. This is precisely how the seed-configuration defect described in §1.1 was caught: no stage's own reported "PASS" would have surfaced it, because each of the two divergent LightGBM implementations was internally self-consistent and passed its own tests. Only a check that cross-referenced two *different* downstream artifacts against each other exposed the disagreement.

## 1.4 Reproducibility boundary conditions

**Question:** Will re-running this pipeline on a different day reproduce the exact numbers in this report?

**Method:** Read `pipeline_config.yaml`'s data-acquisition parameters, and empirically re-run the pipeline twice in the same session to observe whether outputs are byte-identical.

**Evidence:** `data.end_date` is `null` by design ("download through the most recent trading day"). Two full pipeline executions in the same working session, minutes apart, produced canonical datasets with different row counts (3905 rows in the historical snapshot cited earlier in this report vs. 3899 rows in this cycle's regeneration) and entirely different SHA-256 hashes, because Yahoo Finance's auto-adjusted historical prices are retroactively revised as new dividends are declared, and because "most recent trading day" is itself a moving target. Two of the repository's own tests (`test_baseline_mae_matches_committed`, `test_lightgbm_runs_and_produces_predictions`) assert hard-coded historical literals (row count 3905, a specific MAE to six decimal places) and now fail against a fresh run for exactly this reason.

**Interpretation:** This is a known, disclosed limitation, not a defect discovered by accident: the pipeline is correct in the sense that it faithfully reports whatever the specified date range currently contains, but "correct" and "reproducible-to-the-decimal" are different properties, and this project currently has the first without the second. A reader who re-runs this pipeline should expect the same *qualitative* conclusions (see Appendix A.4's fold-level significance results, which held up unchanged across three independent runs including two different historical snapshots) but not byte-identical tables. Anyone requiring the latter must pin `end_date` explicitly before running — this report's own numbers are all stamped with the exact hash and date range they came from for this reason (Appendix A.11).

## 2 Appendix

### A.1 Full feature data dictionary

**Question:** What, precisely, does each of the 11 model features and the target measure, and is any of it computed from information that would not have been available at prediction time  $t$ ?

**Method:** Read `docs/E1-S6_data_dictionary.csv` directly rather than inferring feature semantics from variable names, and cross-reference against E4-S1’s independent point-in-time verification (154 feature values recomputed from a raw frame physically truncated at  $t$ , compared to published values at `rtol=1e-9`, 154/154 matches).

**Evidence:**

| Column                         | Role    | Formula                                                                                                                                   | Window                        | Timestamp semantics                      | se- |
|--------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|------------------------------------------|-----|
| <code>return_1d</code>         | feature | $\text{Close}_t / \text{Close}_{t-1} - 1$                                                                                                 | 1 obs                         | through $t$ only                         |     |
| <code>return_5d</code>         | feature | $\text{Close}_t / \text{Close}_{t-5} - 1$                                                                                                 | 5 obs                         | through $t$ only                         |     |
| <code>return_10d</code>        | feature | $\text{Close}_t / \text{Close}_{t-10} - 1$                                                                                                | 10 obs                        | through $t$ only                         |     |
| <code>return_20d</code>        | feature | $\text{Close}_t / \text{Close}_{t-20} - 1$                                                                                                | 20 obs                        | through $t$ only                         |     |
| <code>volatility_5d</code>     | feature | $\text{std}(\text{return\_1d}[t-4:t], \text{ddof}=1) \cdot \sqrt{252}$                                                                    | 5 obs                         | right-aligned, through $t$               |     |
| <code>volatility_10d</code>    | feature | same, 10-obs window                                                                                                                       | 10 obs                        | right-aligned, through $t$               |     |
| <code>volatility_20d</code>    | feature | same, 20-obs window                                                                                                                       | 20 obs                        | right-aligned, through $t$               |     |
| <code>trend_10d</code>         | feature | $\text{Close}_t / \text{mean}(\text{Close}[t-9:t]) - 1$                                                                                   | 10 obs                        | right-aligned, through $t$               |     |
| <code>trend_20d</code>         | feature | same, 20-obs window                                                                                                                       | 20 obs                        | right-aligned, through $t$               |     |
| <code>trend_60d</code>         | feature | same, 60-obs window                                                                                                                       | 60 obs                        | right-aligned, through $t$               |     |
| <code>volume_ratio_20d</code>  | feature | $\text{Volume}_t / \text{mean}(\text{Volume}[t-19:t])$                                                                                    | 20 obs                        | right-aligned, through $t$               |     |
| <code>forward_return_5d</code> | target  | $\text{Close}_{t+5} / \text{Close}_t - 1$                                                                                                 | 5 future obs                  | label only; never enters $X$             |     |
| <code>regime</code>            | segment | <b>HighVol</b> if $\text{volatility\_20d}_t \geq$ expanding median of prior <code>volatility_20d</code> (min 252 obs), else <b>LowVol</b> | 20d vol + expanding threshold | threshold excludes current & future rows |     |

**Interpretation:** Every feature’s window is right-aligned and the regime threshold is an *expanding*, past-only median rather than a full-sample quantile — E4-S1’s audit specifically demonstrated that a full-sample-median version of the same threshold would disagree with the published labels on 260 rows, i.e. the leakage this construction avoids is not a theoretical concern but one that would visibly change results if introduced. This dictionary is the contract every downstream stage depends on: E2-S2’s and E2-S6’s training scripts all validate their feature list against this file’s manifest at runtime rather than hard-coding column names a second time.

### A.2 Full hyperparameters

**Question:** Were any of the five models’ hyperparameters chosen, adjusted, or searched after seeing out-of-sample results?

**Method:** Check git history for each parameter dictionary (number of commits, whether values ever changed) and grep the entire repository for hyperparameter-search tooling (Optuna, GridSearchCV, RandomizedSearchCV, `.study()`).

**Evidence:**

| Model         | Hyperparameters (fixed a priori, one commit, never modified)                                                                                                                                                                                                                                     |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zero baseline | $\hat{y} = 0$ (no parameters)                                                                                                                                                                                                                                                                    |
| LightGBM      | <code>objective=regression, metric=mae, n_estimators=200, max_depth=4, num_leaves=15, learning_rate=0.05, min_child_samples=30, subsample=0.8, subsample_freq=1, colsample_bytree=0.8, reg_alpha=0.0, reg_lambda=1.0, random_state=42, n_jobs=-1, deterministic=True, force_col_wise=True</code> |
| Random Forest | <code>n_estimators=200, max_depth=4, min_samples_leaf=30, max_features=0.8, random_state=42, n_jobs=-1</code>                                                                                                                                                                                    |
| AdaBoost      | <code>base: max_depth=3, min_samples_leaf=30, random_state=42; ensemble: n_estimators=100, learning_rate=0.05, loss=linear, random_state=42</code>                                                                                                                                               |
| XGBoost       | <code>objective=reg:squarederror, n_estimators=200, max_depth=4, learning_rate=0.05, min_child_weight=30, subsample=0.8, colsample_bytree=0.8, reg_alpha=0.0, reg_lambda=1.0, random_state=42, n_jobs=-1, tree_method=hist</code>                                                                |

**Interpretation:** `random_state` and `n_jobs` for LightGBM are, as of this cycle, no longer literal values duplicated in `pipeline_config.yaml`; they are merged in at call time from `cfg.model.seed` by `pipeline/model.py::lightgbm_model_params()`. This is a direct, mechanical response to the defect in §1.1: the earlier design had exactly one config value (the seed) expressed in two places (a hard-coded literal in `train_lightgbm.py` and an absent key in the YAML), and two places is how it drifted. Three new regression tests now assert, respectively, that the merged parameters include the seed, that they match the standalone script’s reviewed dictionary key-for-key, and that the resulting fit is deterministic on repeated calls — so a future edit to either file that reintroduces a mismatch fails a test immediately rather than propagating silently into a report table.

### A.3 Six fold boundaries

**Question:** Are the six walk-forward folds’ train/test boundaries strictly chronological, with the purge gap actually enforced?

**Method:** Re-derive the fold boundaries independently in E4-S2 from `splits.py` applied to the live canonical dataset’s dates, rather than reading E2-S3’s reported boundaries.

**Evidence:**

| Fold | Train end  | Test start | Test end   | $n_{\text{train}}$ | $n_{\text{test}}$ |
|------|------------|------------|------------|--------------------|-------------------|
| 0    | 2011-01-25 | 2011-02-02 | 2013-08-30 | 1255               | 649               |
| 1    | 2013-08-23 | 2013-09-03 | 2016-03-31 | 1904               | 649               |
| 2    | 2016-03-23 | 2016-04-01 | 2018-10-25 | 2553               | 649               |
| 3    | 2018-10-18 | 2018-10-26 | 2021-05-26 | 3202               | 649               |
| 4    | 2021-05-19 | 2021-05-27 | 2023-12-22 | 3851               | 649               |
| 5    | 2023-12-15 | 2023-12-26 | 2026-08-05 | 4500               | 654               |

**Interpretation:** Every fold satisfies `max(train_date) < min(test_date)` with a gap of exactly 5 trading observations, matching the target horizon — this is checked algebraically (`purge_start_idx = test_start_idx - horizon`, which holds by construction whenever `horizon > 0`), not merely observed empirically. The training window expands rather than rolls (fold 5’s training set is 4500 rows vs. fold 0’s 1255), which is a deliberate choice matching the walk-forward design’s stated rationale (more history available at later folds should not be discarded) but does mean the six folds are not identically-distributed replicates — fold 0’s model sees roughly 5 years of history, fold 5’s sees roughly 18. This asymmetry is relevant when interpreting Appendix A.4’s fold-level significance test, which treats all six folds as exchangeable observations of one underlying effect: that assumption is a simplification, not a certified property of the data.

## A.4 Full results by fold, and whether any of it is distinguishable from noise

**Question:** Do the point-estimate "overall" MAE differences between models (typically 0.0001–0.0006) represent a real effect, or are they within the fold-to-fold noise already visible in each model’s own per-fold metrics?

**Method:** For each model, compute six per-fold MAE improvements over baseline ( $\text{baseline\_MAE}_{\text{fold}} - \text{model\_MAE}_{\text{fold}}$ ), then apply a naive one-sample  $t$ -test across the six folds (`metrics.paired_fold_significance`). This is deliberately the simplest possible test, not a substitute for a properly block-corrected one; its purpose is to catch results that are clearly noise, not to certify significance.

**Evidence:**

| Fold | Zero MAE | LightGBM MAE | RF MAE   | XGB MAE  | AdaBoost MAE |
|------|----------|--------------|----------|----------|--------------|
| 0    | 0.016996 | 0.017678     | 0.016721 | 0.017709 | 0.017141     |
| 1    | 0.013952 | 0.013828     | 0.013613 | 0.013955 | 0.014059     |
| 2    | 0.010483 | 0.010236     | 0.010072 | 0.010230 | 0.010948     |
| 3    | 0.020208 | 0.019979     | 0.019893 | 0.020139 | 0.020486     |
| 4    | 0.018767 | 0.019052     | 0.018601 | 0.019220 | 0.018892     |
| 5    | 0.015418 | 0.015349     | 0.014856 | 0.015308 | 0.015863     |

| Model         | Mean improve. | Naive $t$    | Sign consistent, all 6 folds? | Verdict                   |
|---------------|---------------|--------------|-------------------------------|---------------------------|
| Random Forest | +0.000345     | <b>6.32</b>  | Yes (positive every fold)     | distinguishable from zero |
| AdaBoost      | −0.000261     | <b>−3.94</b> | Yes (negative every fold)     | distinguishable from zero |
| LightGBM      | −0.000050     | −0.33        | No                            | likely noise              |
| XGBoost       | −0.000123     | −0.80        | No                            | likely noise              |

**Interpretation:** This table is the single most consequential piece of evidence Pipeline is responsible for in this report, because it directly determines how RQ1 and RQ3 may honestly be worded. Random Forest’s improvement over baseline is positive in every one of the six folds and its naive  $t$ -statistic (6.32) clears a conventional two-tailed significance threshold even at only 5 degrees of freedom; AdaBoost’s *negative* effect is equally consistent in sign and magnitude ( $t = -3.94$ ) and should be reported as a real, replicated finding, not dismissed as a weak model. LightGBM and XGBoost, by contrast, flip sign across folds and sit within roughly one naive standard error of zero — and this is not a one-off artifact of a single run: across three independent regenerations this cycle (the historical with-seed run, the historical without-seed run, and this cycle’s fresh live-data run), LightGBM and XGBoost’s relative rank versus each other reversed, while Random Forest’s rank (best) and AdaBoost’s rank (worst) never changed. A report that states "LightGBM ranks 4th of 5" as if it were as solid a fact as "Random Forest beats baseline" would be overstating the evidence for the former and, by omission, understating the evidence for the latter and for AdaBoost’s reliable underperformance. The stated caveats on this test (six folds, no correction for possible serial correlation between adjacent folds, folds not identically distributed per Appendix A.3) mean this screening result should raise or lower confidence, not be treated as a final significance certificate.

## A.5 Full model $\times$ regime tables

**Question:** Does any model’s apparent regime-dependence (worse MAE in HighVol than LowVol) reflect a genuine change in predictive skill, or simply that high-volatility returns are larger in magnitude and therefore harder to hit on an absolute-error metric regardless of model quality?

**Method:** Report MAE, correlation, and directional hit rate for all five models across Overall/LowVol/HighVol scopes unconditionally (no metric or scope omitted based on its value), and compare each model’s regime gap against the zero baseline’s own regime gap, since the baseline has no predictive skill to lose.

**Evidence:**

| Model         | Scope   | N    | MAE      | Correlation | Hit rate |
|---------------|---------|------|----------|-------------|----------|
| Zero          | Overall | 3899 | 0.015970 | N/A         | N/A      |
| Zero          | LowVol  | 2216 | 0.011784 | N/A         | N/A      |
| Zero          | HighVol | 1683 | 0.021482 | N/A         | N/A      |
| LightGBM      | Overall | 3899 | 0.016020 | 0.084       | 0.548    |
| LightGBM      | LowVol  | 2216 | 0.011692 | 0.030       | 0.544    |
| LightGBM      | HighVol | 1683 | 0.021718 | 0.096       | 0.553    |
| Random Forest | Overall | 3899 | 0.015625 | 0.105       | 0.597    |
| Random Forest | LowVol  | 2216 | 0.011560 | 0.041       | 0.588    |
| Random Forest | HighVol | 1683 | 0.020978 | 0.116       | 0.608    |
| XGBoost       | Overall | 3899 | 0.016092 | 0.054       | 0.550    |
| XGBoost       | LowVol  | 2216 | 0.011711 | 0.028       | 0.554    |
| XGBoost       | HighVol | 1683 | 0.021862 | 0.058       | 0.545    |
| AdaBoost      | Overall | 3899 | 0.016231 | 0.097       | 0.392    |
| AdaBoost      | LowVol  | 2216 | 0.012019 | -0.044      | 0.407    |
| AdaBoost      | HighVol | 1683 | 0.021777 | 0.121       | 0.371    |

**Interpretation:** Every model’s HighVol MAE is roughly double its LowVol MAE ( $0.0217/0.0118 \approx 1.85$  for LightGBM), but the zero baseline shows almost exactly the same ratio ( $0.0215/0.0118 \approx 1.82$ ) despite having zero skill by construction. This is decisive: the widening MAE in HighVol is overwhelmingly explained by high-volatility returns having larger absolute magnitude, not by any model’s predictions becoming worse in a skill sense. The metric that actually reflects skill — correlation and hit rate — moves the *other* direction for every model except AdaBoost: LightGBM’s correlation rises from 0.030 (LowVol) to 0.096 (HighVol), Random Forest’s from 0.041 to 0.116. If RQ2 is answered using MAE alone, the honest conclusion inverts what a naive reading suggests: these models are not deteriorating in HighVol, they are (mildly) improving on a skill basis, and the MAE increase is a base-rate artifact that would appear even for a model with no information at all.

## A.6 QA checklist / results — case study: the seed-merge defect

**Question:** What did the project’s own QA gates actually catch this cycle, and what would have gone unnoticed without the specific cross-artifact check added this cycle?

**Method:** Run E4-S1 (pre-model leakage gate) and E4-S2 (OOS integrity gate) end-to-end against the live, freshly-regenerated artifacts, and separately verify by direct computation (not by reading either gate’s own report) whether the two gates’ inputs actually agree.

### Evidence:

| Gate                                       | Card  | Verdict     | Notes                                                                                                             |
|--------------------------------------------|-------|-------------|-------------------------------------------------------------------------------------------------------------------|
| Pre-model leak-<br>age & data qual-<br>ity | E4-S1 | <b>PASS</b> | 18/18 tests; one D1 defect (CRLF line-ending hash drift across 11 tracked files) found and fixed during the audit |
| OOS split in-<br>tegrity                   | E4-S2 | <b>PASS</b> | 11/11 checks, 0 violations, 0 warnings (this cycle)                                                               |

E4-S2’s 11 checks: fold chronology; purge-gap effectiveness; no test data in features; no duplicate OOS dates; no in-sample predictions; folds trace to correct test windows; same regime labels / target values / OOS dates across baseline and LightGBM; `results/oos_predictions.csv` matches its declared LightGBM source (not stale); LightGBM MAE agrees between `results/oos_predictions.csv` and E2-S6’s model-ranking table. The last two are new this cycle.

**Interpretation:** The defect this cycle’s new checks were built to catch is worth narrating in full, because it demonstrates exactly why “all gates pass” is not the same claim as “the results are trustworthy.” The pipeline’s LightGBM stage (`pipeline/model.py::run_lightgbm`) constructed `LGBMRegressor(**cfg.model.lightgbm_params)` directly from the YAML dictionary, which had no `random_state` key; the standalone script’s reviewed dictionary did. Both code paths were internally deterministic and passed their own tests — *neither* of the original nine E4-S2 checks would have failed, because none of them compared LightGBM’s own reported numbers across the two downstream tables (E2-S5’s regime table and E2-S6’s ranking table) that both depend on it. The disagreement was only found by directly computing MAE from the raw

prediction CSVs on both sides and discovering they did not match (0.016073 vs. 0.016056 on the historical snapshot). The fix was threefold: (1) merge `cfg.model.seed` into the effective LightGBM parameters at call time so there is one source of truth for the seed, not two; (2) add three regression tests asserting the merge is present, matches the reviewed standalone configuration, and is deterministic; (3) add the two new E4-S2 checks described above, verified in this report by deliberately reintroducing the exact historical stale-MAE value into a test copy of the ranking table and confirming the gate now fails loudly on it (see `test_cross_artifact_check_fails_on_cross_table_mae_mismatch`). Re-running the full corrected pipeline against fresh, live data this cycle reproduced the same qualitative finding (LightGBM does not beat baseline; Random Forest does) under a third, independently-obtained dataset, which is the strongest available evidence that the original finding was not an artifact of the bug being fixed.

## A.7 Pipeline config

```
data:
  ticker: "SPY"
  start_date: "2005-01-01"
  end_date: null # null = download through the most recent trading day
  auto_adjust: true
  trading_days_per_year: 252

features:
  return_windows: [5, 10, 20]
  volatility_windows: [5, 10, 20]
  trend_windows: [10, 20, 60]
  volume_ma_window: 20

target:
  column: "forward_return_5d"
  horizon_trading_days: 5

regime:
  column: "regime"
  volatility_column: "volatility_20d"
  min_historical_observations: 252
  equality_rule: "HighVol"

model:
  seed: 42
  horizon_trading_days: 5
  min_train_size: 1260
  n_folds: 6
  lightgbm_params: {objective: regression, metric: mae, n_estimators: 200,
    max_depth: 4, num_leaves: 15, learning_rate: 0.05, min_child_samples: 30,
    subsample: 0.8, subsample_freq: 1, colsample_bytree: 0.8, reg_alpha: 0.0,
    reg_lambda: 1.0, verbosity: -1, deterministic: true, force_col_wise: true}
```

**Interpretation:** Note what is deliberately absent from `lightgbm_params` here: `random_state` and `n_jobs`. This is not an oversight left uncorrected — it is the residue of the defect discussed in Appendix A.6, kept as-is in the YAML intentionally, with the fix applied instead at the code layer (`pipeline/model.py:lightgbm_model_params`) so that `cfg.model.seed` remains the single source of truth rather than being duplicated as a second literal inside this file. A future editor who adds a `random_state` key directly to this YAML block would create exactly the two-sources-of-truth condition that caused the original defect, and should instead change `model.seed` above.

## A.8 Package versions

**Evidence:**



| Package      | Pinned (requirements.txt) | Actually used this cycle |
|--------------|---------------------------|--------------------------|
| numpy        | 2.5.2                     | 2.5.2                    |
| pandas       | 3.0.5                     | 3.0.5                    |
| lightgbm     | 4.6.0                     | <b>4.7.0</b>             |
| scikit-learn | 1.9.0                     | 1.9.0                    |
| xgboost      | 3.4.1                     | 3.4.1                    |
| Python       | —                         | 3.14.7                   |

**Interpretation:** This cycle’s verification environment installed `lightgbm` without pinning and received 4.7.0, one minor version ahead of the repository’s pinned 4.6.0. The qualitative results in Appendix A.4 were not observed to be sensitive to this difference, but the discrepancy itself is evidence for a point raised independently in this project’s own prior audits: `requirements.txt` pins exact versions but there is no lockfile or container digest, so `pip install -r requirements.txt` today would resolve to the pinned versions, but a compromised or silently-altered release at the same version number would not be caught, and an environment built without strictly following that file (as this verification cycle’s environment was, for expedience) can silently drift. Future verification cycles should install via `pip install -r requirements.txt` exactly, not via ad hoc package installation.

## A.9 Repository tree (top level)

```
data/ (raw + processed, gitignored, regenerated by pipeline)
docs/ (audit reports, data dictionary, this report)
E2-S1_Baseline_Zero_Predictor/
E2-S2_Train_Minimal_LightGBM_Regressor/
E2-S3_Leakage_Safe_Walk_Forward_Validation/
E2-S4_Generate_Canonical_OOS_Prediction_Table/
E2-S5_Evaluate_Overall_LowVol_HighVol_Performance/
E2-S6_Multi_Model_Comparison/
E2-S7_OOS_Error_Analysis/
E3-S2_Data_Model_Integration_Flow/ (unified pipeline: config.py, model.py, run_pipeline.py)
E4-S2_OOS_Split_Integrity_Gate/
results/ (canonical OOS table + manifest)
tests/
pipeline_config.yaml
pipeline_manifest.json (aggregated stage contracts, regenerated per run)
requirements.txt
```

## A.10 Reproduction commands

**Question:** Can a third party reproduce this report’s qualitative conclusions from a clean checkout?

**Method:** Execute the full command sequence below from a clean state, immediately before writing this Appendix, and record every command actually run rather than an idealized version of it.

**Evidence:**

```
pip install -r requirements.txt

# Full pipeline: data_foundation -> baseline -> lightgbm -> validation
# -> canonical_oos -> regime_evaluation
cd E3-S2_Data_Model_Integration_Flow
python -m pipeline.run_pipeline --force

# Follow-on comparisons (consume results/oos_predictions.csv)
cd ..
python E2-S6_Multi_Model_Comparison/train_additional_models.py
python E2-S6_Multi_Model_Comparison/compare_all_models.py
python E2-S7_OOS_Error_Analysis/analyze_oos_errors.py

# Independent verification gate (must PASS before citing any number above)
```

```
python E4-S2_OOS_Split_Integrity_Gate/audit_oos_split_integrity.py
```

```
# Test suites
python -m pytest E2-S1_Baseline_Zero_Predictor/tests/ \
  E2-S2_Train_Minimal_LightGBM_Regressor/tests/ \
  E2-S3_Leakage_Safe_Walk_Forward_Validation/tests/ \
  E4-S2_OOS_Split_Integrity_Gate/tests/ \
  E3-S2_Data_Model_Integration_Flow/pipeline/tests/ \
  E2-S7_OOS_Error_Analysis/tests/ -q
```

**Interpretation:** This sequence was run in full this cycle and produced the E4-S2 PASS result and the numbers reported throughout this Appendix; it is not a hypothetical procedure. Two tests are expected to fail on a fresh run for the reason documented in §1.4 (`end_date: null`): `test_baseline_mae_matches_committed` and `test_lightgbm_runs_and_produces_predictions` assert historical literals that a live data pull will not reproduce exactly. A third party should treat those two failures as expected, not as evidence the reproduction failed, provided the E4-S2 gate itself still reports PASS.

## A.11 Manifest / hash information

**Evidence:**

| Artifact                                                  | SHA-256 (first 24 chars)                                 |
|-----------------------------------------------------------|----------------------------------------------------------|
| pipeline_config.yaml (config_hash)                        | 7d4b36aa712c99f54340809f...                              |
| data/processed/E1-S6_canonical_modeling_dataset.csv       | 94647ea09890da14aa980a83...                              |
| docs/E1-S6_data_dictionary.csv                            | 3e478195e58d1a4ceae1d53b...                              |
| E2-S1.../baseline_zero_oos_predictions.csv                | df5ee93f72a322aebc4b46f8...                              |
| E2-S2.../lightgbm_oos_predictions.csv                     | e673e804cdafe75767de6710...                              |
| E2-S3.../fold_boundary_audit.csv                          | 8922c5b1461913ffd5c2e3d5...                              |
| results/oos_predictions.csv                               | dfe3655a2495767ec9458691...                              |
| oos_predictions_manifest.json → source_predictions_sha256 | e673e804cdafe75767de6710...                              |
|                                                           | (matches<br>lightgbm_oos_predictions.csv<br>— not stale) |

**Interpretation:** The last row is the exact check that would have caught the seed-merge defect after the fact even without recomputing MAE directly: the declared source hash inside `oos_predictions_manifest.json` must equal the live hash of the file it claims to be built from. This equality is now asserted automatically by E4-S2 (Appendix A.6) rather than needing to be checked by hand for every report cycle.

## A.12 Additional plots

**Evidence:**

| Plot                                           | Source data                                         |
|------------------------------------------------|-----------------------------------------------------|
| Fig. 2 — SPY price/return context              | data/processed/E1-S6_canonical_modeling_dataset.csv |
| Fig. 3 — 20D volatility + regime timeline      | same, volatility_20d + regime columns               |
| Fig. 4 — Walk-forward + purge diagram          | Appendix A.3 fold boundary table                    |
| Fig. 5 — Overall OOS MAE comparison (5 models) | E2-S6.../output/all_models_overall_ranking.csv      |
| Fig. 6 — LowVol vs. HighVol performance        | Appendix A.5 table                                  |
| New: per-fold improvement bar chart            | Appendix A.4 fold-level significance table          |

**Interpretation:** The last row is not on the guideline’s original recommended list, and is the one plot this project most needs but does not yet have: a bar chart of each model’s six per-fold improvements over baseline, grouped by

model, would make Random Forest’s and AdaBoost’s consistent-sign bars visually distinguishable from LightGBM’s and XGBoost’s sign-flipping ones at a glance, in a way that a single ”overall MAE” bar chart (Fig. 5) actively obscures by collapsing exactly the variability that Appendix A.4 shows is the whole story.